DELTATRACE: EFFICIENT SIGNED ATTRIBUTION FOR REASONING LANGUAGE MODELS

Anonymous authors
Paper under double-blind review

ABSTRACT

Explaining a reasoning response involves tracing both the evidence an input supplies and the decisions that govern its use. We introduce DELTATRACE, a method that assigns signed credit to input tokens by following a finite intervention through the model. The original and reference inputs define a change in the log-probability of the same complete response. DELTATRACE allocates this change through analytic operator rules and composes them in one reverse traversal. The resulting source contributions sum to the response-score difference, giving supporting and opposing evidence a common, additive measure. Its central mechanism follows content and control together: attention propagates both transported values and changes in their selection, while gated delta memory propagates stored content alongside the keys, queries, and gates governing retention, writing, and reading. Analytic normalization preserves competition among attention sources. The implementation supports FlashAttention-style tiling and Flash Linear Attention chunked computation, with auxiliary storage linear in sequence length at fixed model dimensions and chunk size. Across 1,243 released Qwen3-8B examples, DELTATRACE achieves the lowest RISE on 9 of 13 tasks and the lowest MAS on 11 among seven methods, and the highest recovery on all six NIAH tasks. NIAH recovery gains over FlashTrace range from 3.96 to 19.60 percentage points. Separate Qwen3.5-9B development measurements demonstrate the hybrid-model implementation and a 35.3% throughput increase from real batching and GPU checkpoints. The counterfactual formulation also motivates studying finite credit propagation for policy learning, where exactly averaged action-level return contrasts recover the advantage signal.

1 INTRODUCTION

Input attribution explains how the words in a prompt and its context contribute to a model's response. For reasoning language models, this means tracing evidence through the computations that select, combine, and retain it (Ferrando & Voita, 2024). Figure 1 illustrates the task with a passage containing a playwright, William Shakespeare, and a composer, William Walton. The names supply candidate information; the phrase "the author of the play" determines which role the answer calls for. The observed attribution assigns positive credit to the playwright's name and negative credit to the composer's. This example brings together three parts of an explanation: what information a source supplies, how it governs the use of information, and the direction of its contribution to the response.

The quantity being explained gives these contributions their meaning. Reference-based attribution assigns prediction differences to input features through path integration or propagated activation differences (Sundararajan et al., 2017; Shrikumar et al., 2017). Relevance propagation develops conservation rules for attention and selective state-space computation (Ali et al., 2022; Achtibat et al., 2024; Rezaei Jafari et al., 2024); representation decomposition exposes how source information mixes across transformer layers and recurrent states (Modarressi et al., 2023; Pitorro & Treviso, 2025). At the response level, FlashTrace aggregates and recursively traces importance for multi-token spans (Pan et al., 2026). We bring the reference-difference perspective to complete reasoning responses: changing a reference input into the original context defines a finite intervention, and the resulting change in the fixed response's log-probability is the effect to be allocated. Content and the controls governing its use then share one signed measure of contribution.

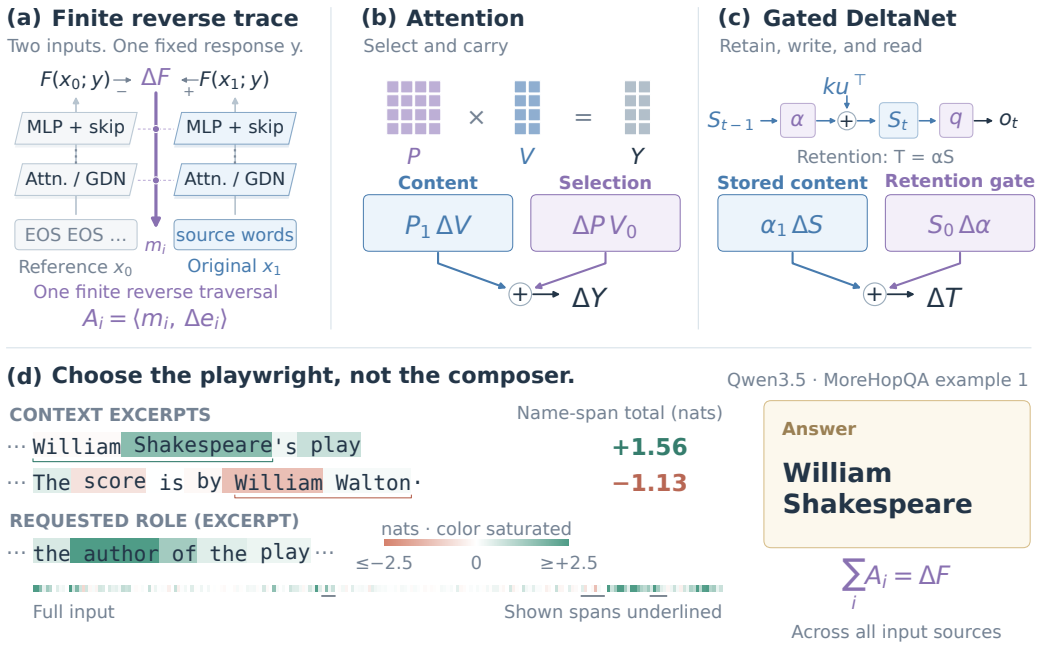

Figure 1: **DeltaTrace follows both content and control.** (a) Original and EOS-reference executions define a fixed-response score change; finite reverse propagation assigns source contributions. (b,c) Attention and gated memory share a content/control allocation. The memory example expands retention, $T = \alpha S$, with S the previous state. The operators are schematic. (d) Actual Qwen3.5 scores from MoreHopQA development example 1 assign positive credit to the playwright's name and negative credit to the composer's name. Backgrounds show token scores; annotations sum only the indicated name spans. One linear color scale is explicitly saturated at ± 2.5 nats. Ellipses mark omitted text. The requested role and identified playwright are shown; attribution targets the complete stored response plus EOS. Conservation refers to all input sources.

The central challenge is that content and control change together. An attention input can alter both the value being carried and the weights that select it; softmax couples those weights across an entire row. Gated memory adds a temporal interaction: each write updates a state assembled from earlier inputs, and later queries and gates determine how that state reaches the response (Yang et al., 2025). Allocating these joint changes requires local rules that remain consistent with the same response-score difference when composed across layers and time. The rules also need an efficient computational form: attention connects pairs of positions, while memory propagates interactions through a sequence of states.

We present DELTATRACE, built on the idea that a finite response change can be traced by composing finite changes at each operation. Two executions share the same model and fixed response, with the source input set to the reference or original context. Local rules express each observed output difference in terms of its input differences; one reverse traversal composes them into source contributions. The finite chain rule gives $\sum_i A_i = \Delta F$, tying every score to an additive share of the intervention's total effect. This construction gives the explanation both a counterfactual meaning and a direction: positive and negative credit describe supporting and opposing contributions to the response-score change. Their magnitudes share a common unit, and token scores add directly into word or passage scores.

The operator rules follow how the model uses evidence. In attention, content changes travel with the original execution's selection weights, while selection changes act on the reference content. An analytic softmax rule propagates competition among sources. In gated delta memory, the same principle follows stored content and the keys, queries, and gates governing retention, writing, and reading. A source can therefore receive credit both for the information it provides and for directing

its use, as the name and requested role illustrate in Figure 1. These rules also fit the model's accelerated computation: attention uses FlashAttention-style tiled reductions (Dao et al., 2022), and the linear-attention path uses Flash Linear Attention (FLA) chunk-state adjoints, batched products, and scans (Yang & Zhang, 2024). Processing interactions within tiles and chunks gives auxiliary storage linear in sequence length at fixed model dimensions and chunk size.

The paper develops three connected contributions:

- **Signed credit grounded in a finite intervention.** A complete response-score difference defines the attribution target, and a finite chain rule conserves its allocation across input sources (Section 2; Appendix A.1).
- **A shared content-and-control mechanism.** Analytic attention normalization and gated delta recurrence trace transported evidence together with the decisions that select and retain it (Sections 2.3–2.4; Appendix A).
- **Attribution through accelerated attention kernels.** Tiled dense-attention propagation and native FLA chunk-state operations support attention and hybrid models with sequence-linear auxiliary storage (Section 3).

A complete evaluation on 1,243 released Qwen3-8B examples connects these contributions to evidence recovery and deletion faithfulness. Among seven methods, DELTATRACE obtains the lowest RISE on 9 of 13 tasks, the lowest MAS on 11, and the highest recovery on all six NIAH tasks (Tables 1–3). Paired implementation measurements report complete attribution costs, and separate Qwen3.5-9B development results exercise the shared rules through hybrid attention and memory (Section 4). The same counterfactual perspective suggests a direction for policy learning: averaging action-level return contrasts over a policy reference yields its exact advantage and hence the same expected policy-gradient signal as exact action values (Sutton et al., 1999; Foerster et al., 2018). Appendix F states this connection and develops the prospect of estimating learning credit through finite propagation.

2 DELTATRACE

2.1 WHAT CONTRIBUTES TO A RESPONSE

Identifying a playwright in a passage draws on both a person's name and the role specified by the question. In Figure 1, "William Shakespeare" supplies the requested name, while "author of the play" specifies which person to use. The nearby composer provides a contrasting role in the same context. DELTATRACE traces the contributions of these sources through the model, connecting the response to words that supply evidence and guide its use.

The explanation concerns a fixed response y. Its score is the sum of the response tokens' log-probabilities,

$$F(x; y) = \sum_{t \in \mathcal{T}} \log p_\theta(y_t \mid x, y_{<t}), \tag{1}$$

where $\mathcal{T}$ selects the response positions. DELTATRACE compares the original input x_1 with a reference x_0, keeping the response fixed. This comparison defines the change being explained: $\Delta F = F(x_1; y) - F(x_0; y)$. Setting the source embeddings to these two inputs, with model parameters and response held fixed, defines the intervention whose total effect is allocated. In our evaluation, the reference replaces eligible source tokens with EOS. Each source receives a signed share of this joint input effect under the operator rules below. A positive share contributes toward the increase in the response score; a negative share contributes in the opposite direction.

2.2 FOLLOWING EVIDENCE BACKWARD

DELTATRACE first runs the model on the original and reference inputs, recording how its internal representations change. It then follows the response backward through the computation. At each operation, it distributes the contribution arriving at the output among the inputs that produced that output. Attention sends contributions to the positions it reads; memory carries them through earlier

writes; residual connections divide them among branches. Contributions that reach the same input are added together.

Each local rule accounts for the change across the two executions. For a nonlinear activation, this means relating its observed output change to its input change across that interval. For a weighted sum, the rule follows the weights and the changes in the vectors being combined. Composing these rules gives the coefficient m_i of each source's embedding change Δe_i, yielding

$$A_i = \langle m_i, \Delta e_i \rangle . \tag{2}$$

Thus a source receives credit through every computational path connecting it to the response. The sign is carried through those paths, so supporting and opposing contributions can meet and cancel. The local accounting preserves the total response change:

$$\sum_i A_i = F(x_1; y) - F(x_0; y). \tag{3}$$

This places the source scores on a common scale. Appendix A.1 gives the finite chain rule and conservation theorem.

2.3 What attention carries and selects

An attention head combines the content at source positions using weights chosen by queries and keys. A source can therefore affect the output by changing what is carried, how strongly it is selected, or both. In the playwright example, the value representation carries information about the name, while the attention weights determine how much of that information reaches a later position. DeltaTrace follows both routes to the response.

For attention probabilities P and values V, the output is $Y = PV$. Its change separates into

$$\Delta Y = \underbrace{P_1 \Delta V}_{\text{content change}} + \underbrace{\Delta P\, V_0}_{\text{routing change}} . \tag{4}$$

The content term uses the attention chosen for the original input. A change in a source's value therefore travels with the weight that the model assigns to it in that execution. The routing term measures the effect of redistributing attention over the reference values. When content and selection change together, their interaction is assigned to the content term. These two terms give a complete account of the attention output change while retaining the contribution of selection itself.

Attention also couples the sources within each row: increasing the share assigned to one source redistributes weight among the others. DeltaTrace carries this competition backward through normalization. If u_i is the coefficient arriving at an attention probability, its score coefficient is

$$m_{z,i} = \ell_i (u_i - \bar{u}_\ell), \qquad \bar{u}_\ell = \frac{\sum_j \ell_j u_j}{\sum_j \ell_j}. \tag{5}$$

The positive weight ℓ_i is the logarithmic mean of that source's probabilities in the two executions. Subtracting the weighted mean expresses each source's contribution relative to the alternatives in the same row. Query and key coefficients then carry this routing contribution to the representations that selected the sources. The same normalization construction connects the response's log-probability to its output logits. Appendix A.2 specifies the logarithmic mean and derives the finite identity.

2.4 What memory keeps and updates

In a hybrid model, evidence can also reach a response through a memory state. Information written at an earlier position can survive several updates and be read by a later query. For the playwright example, the relevant name may be written when the model processes the source sentence and read again at a later position. The explanation therefore follows both the stored content and the gates that determine its survival and use.

Gated delta memory combines retained information with a new write (Yang et al., 2025):

$$S_t = \underbrace{\alpha_t S_{t-1}}_{\text{retained evidence}} + \underbrace{k_t u_t^\top}_{\text{new write}} . \tag{6}$$

The retention gate α_t controls how much of the previous state remains. The write vector u_t updates the content associated with key k_t: it is the difference between the incoming value and the value read from retained memory, scaled by a write gate. A query reads the resulting state to produce the token's memory output.

DELTATRACE reverses this sequence of reading, writing, and retention. At a read, contributions pass to the stored content and to the query that retrieves it. At a write, they pass to the incoming value, the retrieval key, and the write gate. Along the retained state, they continue to earlier memory contents and to the retention gate. Each step uses the same content-and-control allocation as attention: changing content is weighted by the original execution's control, and control changes receive their own contribution. This lets an earlier source receive a signed contribution through the memory operations that connect it to the response. Appendix A.4 gives the complete recurrence.

Shared rules across models. The same operator uses the same propagation rule in Qwen3 and Qwen3.5. Attention values use Equation 4; query–key products and SwiGLU divide their interaction symmetrically; memory and output gates use the content-and-control allocation described above. Linear layers, residual additions, and normalization complete the traversal through each architecture. Appendix A.3 collects the fixed rules and their formulas.

3 EFFICIENT COMPUTATION

Accumulating attention contributions in tiles. The attention mechanism produces a contribution for each source vector and routing score. DELTATRACE accumulates these contributions a tile at a time using the FlashAttention framework (Dao et al., 2022). Within each tile, it reconstructs the attention probabilities, computes the required weighted sums, and adds the result to the source coefficients. The tile can then be reused. The quantities carried across tiles are vectors and row statistics, which grow linearly with the number of tokens.

The weighted mean in Equation 5 supplies the information shared across an attention row. A first sweep accumulates this mean and its normalizer. Subsequent sweeps use those statistics to propagate contributions to queries, keys, and values. This organization puts the content and routing computations into a fixed number of tiled sweeps. For sequence length T and head dimension d, auxiliary attention storage is $O(Td)$ and dense attention arithmetic is $O(T^2d)$ per head.

Following memory through chunks. Memory passes information between consecutive chunks through a boundary state. DELTATRACE follows the contribution to that state backward, then distributes it among the reads, writes, and gates inside each chunk. The implementation supports the accelerated linear-attention path through Flash Linear Attention (FLA) (Yang & Zhang, 2024): native chunk-state adjoints propagate the boundary coefficients, and batched matrix products distribute finite contributions inside each chunk. An associative scan accumulates the retention contribution. At fixed state dimensions and chunk size, the stored token contributions and boundary states grow linearly with sequence length. Appendix B gives the tensor accounting and execution details.

Complete attribution. The full procedure consists of the two model executions, layer replay, backward contribution propagation, and transfers. Layer replay supplies the activations needed by the current layer, while the tiled attention and chunked memory computations accumulate its input contributions. Together, these steps apply the same attribution method to attention and hybrid models. The evaluation measures their combined latency, memory use, and throughput.

4 EVALUATION

Tasks and fixed responses. We evaluate Qwen3-8B on all 1,243 examples in the authors' `table1-data-v1` release accompanying FlashTrace (Pan et al., 2026).[1] The selection contains ten RULER tasks with 100 examples each, HotpotQA with 48, MATH with 100, and MoreHopQA with 95. Deletion faithfulness scores the complete released response plus EOS, which is also the attribution target for DELTATRACE. Its input formatting, eligible positions, gold mapping, and FP16

[1] https://github.com/wbopan/flashtrace/releases/tag/table1-data-v1

Table 1: RISE ↓ on the complete released Qwen3-8B tasks. FT is FlashTrace and DT is DELTATRACE. The five other baselines and FT use published aggregates (Pan et al., 2026); DT uses signed ranking. n is the released task size. Bold marks the lowest observed mean across all seven methods.

Task	n	Perturbation	REAGENT	CLP	IFR	AttnLRP	FT	DT
MQ-Q2	100	0.0945	0.1174	0.0977	0.0747	0.1955	0.0681	**0.0645**
MQ-Q4	100	0.2389	0.2599	0.2534	0.1147	0.2628	**0.1134**	0.1310
MQ-Q8	100	0.4986	0.4865	0.5096	0.3709	0.3766	**0.3518**	0.3757
MV-V2	100	0.1343	0.1876	0.1559	0.0691	0.1403	0.0685	**0.0521**
MV-V4	100	0.1862	0.2107	0.2166	0.0726	0.1934	0.0704	**0.0525**
MV-V8	100	0.3514	0.3685	0.3929	0.2051	0.2852	0.1826	**0.1022**
VT-H2-C3	100	0.3844	0.4378	0.3742	0.1609	0.3191	0.1316	**0.0900**
VT-H4-C1	100	0.3538	0.3971	0.3277	**0.1018**	0.3239	0.1102	0.1099
VT-H6-C1	100	0.4584	0.4860	0.4225	0.1245	0.3378	0.1224	**0.1145**
VT-H10-C1	100	0.4663	0.4953	0.4510	0.1526	0.3573	**0.1426**	0.1448
HotpotQA	48	0.1333	0.1445	0.1013	0.0745	0.1548	0.0330	**0.0281**
MATH	100	0.3802	0.3878	0.3616	0.3537	0.3684	0.3484	**0.2029**
MoreHopQA	95	0.2489	0.2652	0.2487	0.1455	0.2857	0.1280	**0.0697**

Table 2: MAS ↓ on the same released tasks and seven methods as Table 1. DT uses its positive source contributions. Bold marks the lowest observed mean.

Task	n	Perturbation	REAGENT	CLP	IFR	AttnLRP	FT	DT
MQ-Q2	100	0.1442	0.1966	0.1663	0.1397	0.3261	0.1628	**0.0925**
MQ-Q4	100	0.3270	0.3574	0.3203	**0.1770**	0.4509	0.1935	0.1825
MQ-Q8	100	0.7093	0.6942	0.6566	0.4600	0.6020	**0.4270**	0.5449
MV-V2	100	0.1870	0.2756	0.2159	0.1343	0.2290	0.1570	**0.0810**
MV-V4	100	0.2436	0.2905	0.2804	0.1416	0.3248	0.1604	**0.0835**
MV-V8	100	0.4580	0.4938	0.5114	0.2746	0.4752	0.2476	**0.1442**
VT-H2-C3	100	0.5513	0.6680	0.4900	0.2309	0.5214	0.1872	**0.1157**
VT-H4-C1	100	0.5168	0.6031	0.4199	0.1479	0.5475	0.1655	**0.1451**
VT-H6-C1	100	0.6837	0.7453	0.5648	0.1727	0.5723	0.1787	**0.1483**
VT-H10-C1	100	0.7013	0.7405	0.5967	0.2011	0.5915	0.1982	**0.1851**
HotpotQA	48	0.2203	0.2351	0.1895	0.1655	0.2493	0.1277	**0.0627**
MATH	100	0.5202	0.5358	0.4907	0.4897	0.5244	0.4456	**0.2888**
MoreHopQA	95	0.3470	0.3679	0.3478	0.2277	0.4579	0.2049	**0.1186**

eager scoring follow the released code. The frozen `clean-v1` method uses the shared rules in Section 2; the faithfulness comparison includes published baseline aggregates. VT and HotpotQA recovery use the separate re-evaluation specified below.

Source recovery and deletion faithfulness. NIAH recovery uses the top 10% of released eligible tokens. VT reports body-token Recall at budgets of 10%, 10%, 20%, and 30% for H2, H4, H6, and H10. HotpotQA reports supporting-fact Recall with a 10% body-token budget. Faithfulness uses the authors' 20-step deletion procedure: RISE ranks signed contributions and MAS ranks their positive part; lower is better. Appendix C specifies the recovery protocols.

Complete task results. Across seven methods, DELTATRACE achieves the lowest RISE on 9 of 13 tasks and the lowest MAS on 11 (Tables 1 and 2). Against FlashTrace, the corresponding improvements cover 10 and 12 tasks. The largest absolute RISE reduction occurs on MATH, from 0.3484 to 0.2029. Recovery is highest on all six NIAH tasks, with gains over FlashTrace of 3.96–19.60 percentage points. At the stated VT budgets, DELTATRACE reaches an 84.18% macro Recall versus 76.57% for FlashTrace K3. On HotpotQA, IFR has the highest mean, 75.87%, versus 72.05% for DELTATRACE (Table 3). These comparisons describe observed means.

Hybrid-model development results. A separate paired development set contains the first eight MQ-Q2 and eight MoreHopQA examples for each model. On Qwen3.5-9B, recovery rises from

Table 3: Evidence recovery (%, higher is better) at the listed token budgets. NIAH/VT report token Recall; HotpotQA reports supporting-fact Recall. VT macro averages its four rows. For VT/HotpotQA, FT uses K3 and † denotes amended AttnLRP (Appendix C). Bold marks the highest observed mean.

Task	n	Budget	Perturbation	REAGENT	CLP	IFR	AttnLRP†	FT	DT
MQ-Q2	100	10%	39.05	24.35	39.85	47.10	21.49	48.34	**66.76**
MQ-Q4	100	10%	9.04	8.49	8.58	32.84	20.42	41.33	**47.83**
MQ-Q8	100	10%	0.97	0.47	0.84	1.19	7.63	7.50	**13.24**
MV-V2	100	10%	25.48	18.02	20.67	57.50	25.44	55.56	**75.16**
MV-V4	100	10%	16.05	15.60	14.61	45.24	24.25	51.59	**57.52**
MV-V8	100	10%	8.02	7.37	7.29	17.87	15.88	20.36	**24.32**
VT-H2-C3	100	10%	26.69	26.66	29.10	81.16	83.37	72.41	**85.62**
VT-H4-C1	100	10%	18.86	17.58	20.53	70.09	70.54	68.32	**71.33**
VT-H6-C1	100	20%	33.57	32.47	33.85	83.74	84.57	82.15	**92.05**
VT-H10-C1	100	30%	40.70	42.48	44.04	82.66	77.33	83.41	**87.74**
VT macro	400	mixed	29.96	29.80	31.88	79.41	78.95	76.57	**84.18**
HotpotQA	48	10%	37.50	34.72	42.88	**75.87**	74.83	66.67	72.05

63.92% to 73.35%; MAS decreases on both tasks. RISE is 0.1115 versus 0.0942 on MQ-Q2 and 0.1688 versus 0.2356 on MoreHopQA. These results exercise the same finite rules through the hybrid model's attention and gated-memory paths. Table 5 gives the complete development comparison for both models.

Measured attribution cost. Figure 2 reports complete attribution calls, including input preparation, endpoint capture, layer replay, propagation, and returning scores. Loading and warmup are recorded separately. On the 16-example Qwen3-8B implementation benchmark, the sum of per-example median latencies is 11.40 seconds for DELTATRACE and 12.16 seconds for one-hop FlashTrace; maximum allocated memory is 18.76 and 21.49 GB, respectively. On Qwen3.5-9B, real batches of two examples with GPU checkpoints reduce the mean time per 16 examples from 16.68 to 12.33 seconds, increasing throughput by 35.3%. The corresponding allocated-memory peaks are 21.64 and 23.88 GB. Appendix D records the benchmark snapshots, repetition protocol, longer-input tiling measurements, and additional replay-retention results.

Signed evidence in concrete responses. Figures 4 and 5 show the first released example from each development task with the same fixed response evaluated by both models. The retrieval view connects two source keys to their numbers and to the query naming those keys. The multi-hop view follows the context linking a voice actor to a film and its director, whose surname determines the answer. Teal and coral retain the direction of each token's contribution to the complete response score. Aligned context and question excerpts show the evidence, and full-input strips locate it in the surrounding context. The accompanying deletion curves compare each model's positive-ranking DT attribution with its own one-hop FlashTrace baseline, measuring the normalized log-likelihood of the entire fixed response plus EOS.

5 RELATED WORK

Reference differences and conserved relevance. Integrated Gradients integrates input sensitivities along the straight-line path from a baseline to the input, and DeepLIFT composes activation differences into source contributions (Sundararajan et al., 2017; Shrikumar et al., 2017). Conservative Propagation and AttnLRP develop relevance rules for transformer computations (Ali et al., 2022; Achtibat et al., 2024). DELTATRACE builds on the reference-difference view with analytic finite rules for attention normalization and gated delta memory. Its content and control allocations compose into a conserved difference in the log-probability of a complete response, connecting the attribution's numerical meaning to the operators carrying it.

Representation decomposition and response tracing. ALTI aggregates token interactions across layers, while DecompX propagates decomposed representations to the prediction head (Ferrando

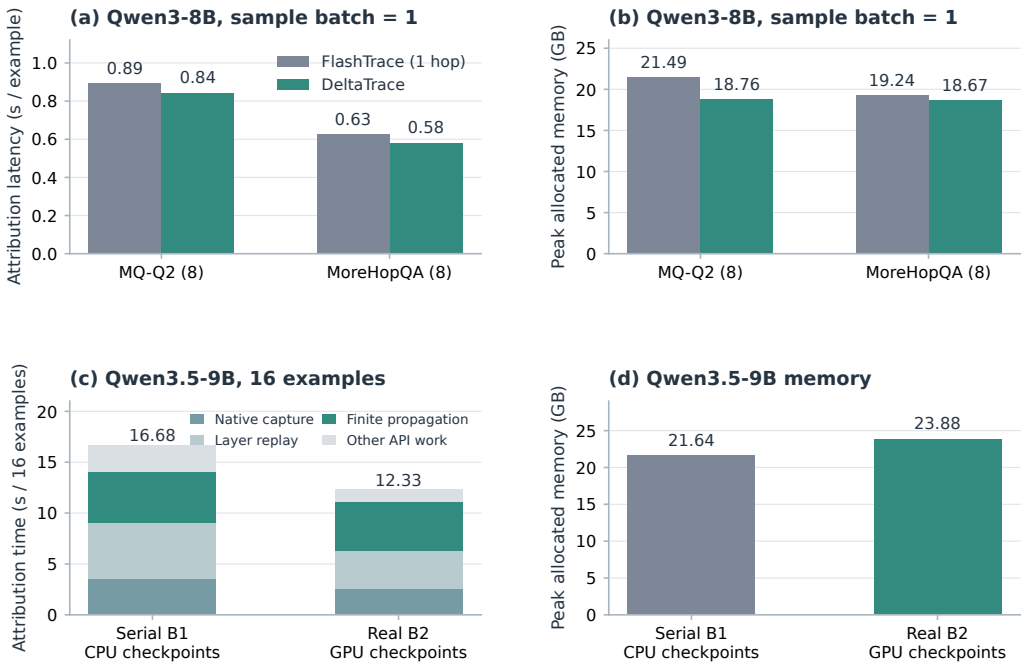

Figure 2: **Measured cost of complete attribution.** (a,b) Qwen3-8B, eight MQ-Q2 and eight MoreHopQA examples: mean of three-run per-example median latencies and maximum allocated memory. FlashTrace uses one recursive hop. (c,d) Qwen3.5-9B, the same 16 development examples: two paired warm rounds comparing serial B1 with CPU checkpoints against real B2 with GPU checkpoints. Stacks show recorded API stages, and each memory maximum includes resident model weights. B denotes real examples per call; each example has two endpoints. GB denotes 10^9 bytes. The Qwen3 and Qwen3.5 panels are separate implementation benchmarks.

et al., 2022; Modarressi et al., 2023). Information Flow Routes traces influential computations through a model (Ferrando & Voita, 2024), and FlashTrace aggregates target spans and recursively follows their source importance (Pan et al., 2026). DELTATRACE traces a scalar response-score difference through both the transported content and its input-dependent selection. All response positions enter one target, and the finite reverse coefficients combine their contributions at the input sources.

Recurrent interpretation and efficient execution. MambaLRP brings relevance conservation to selective state-space models, and LaTIM decomposes token interactions in Mamba-1 and Mamba-2 (Rezaei Jafari et al., 2024; Pitorro & Treviso, 2025). DELTATRACE treats the delta-rule memory of Gated Delta Networks (Yang et al., 2025), including the finite contributions of retrieval, correction, retention, and writing. Its rules share a content-and-control allocation with attention. FlashAttention tiling (Dao et al., 2022) and FLA's accelerated linear-attention computation (Yang & Zhang, 2024) supply the execution structures: row reductions, chunk-state adjoints, and scans realize these contributions with sequence-linear auxiliary storage.

6 CONCLUSION

DELTATRACE explains a complete response through signed credit for a finite input intervention. Its analytic rules follow both the evidence being transported and the attention or memory controls governing its use. Contribution conservation connects these local mechanisms to the total response-score change, and FlashAttention-style tiling and native FLA operations give the attribution sequence-linear auxiliary storage. The full Qwen3 evaluation links this construction to recovery and deletion faithfulness across 13 tasks; paired hybrid-model development and implementation

benchmarks extend the evidence to gated memory and measured attribution cost. The counterfactual view also opens a path toward studying reward-level credit propagation as an estimator of policy-learning signals.

AI USE STATEMENT

AI assistance supported conceptual development, mathematical derivations, research code, experimental design and analysis, literature retrieval, figure design, manuscript organization, and language editing. The manuscript uses the released evaluation data and model-generated trajectories specified in the evaluation protocol.

REFERENCES

Reduan Achtibat, Sayed Mohammad Vakilzadeh Hatefi, Maximilian Dreyer, Aakriti Jain, Thomas Wiegand, Sebastian Lapuschkin, and Wojciech Samek. AttnLRP: Attention-aware layer-wise relevance propagation for transformers. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pp. 135–168. PMLR, 2024. URL https://proceedings.mlr.press/v235/achtibat24a.html.

Ameen Ali, Thomas Schnake, Oliver Eberle, Grégoire Montavon, Klaus-Robert Müller, and Lior Wolf. XAI for transformers: Better explanations through conservative propagation. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162, pp. 435–451. PMLR, 2022. URL https://proceedings.mlr.press/v162/ali22a.html.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *arXiv preprint arXiv:2205.14135*, 2022. URL https://arxiv.org/abs/2205.14135.

Javier Ferrando and Elena Voita. Information flow routes: Automatically interpreting language models at scale. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 17432–17445. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.emnlp-main.965. URL https://aclanthology.org/2024.emnlp-main.965/.

Javier Ferrando, Gerard I. Gállego, and Marta R. Costa-jussà. Measuring the mixing of contextual information in the transformer. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 8698–8714. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.emnlp-main.595. URL https://aclanthology.org/2022.emnlp-main.595/.

Jakob Foerster, Gregory Farquhar, Triantafyllos Afouras, Nantas Nardelli, and Shimon Whiteson. Counterfactual multi-agent policy gradients. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, pp. 2974–2982, 2018. doi: 10.1609/aaai.v32i1.11794. URL https://ojs.aaai.org/index.php/AAAI/article/view/11794.

Ali Modarressi, Mohsen Fayyaz, Ehsan Aghazadeh, Yadollah Yaghoobzadeh, and Mohammad Taher Pilehvar. DecompX: Explaining transformers decisions by propagating token decomposition. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 2649–2664. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.acl-long.149. URL https://aclanthology.org/2023.acl-long.149/.

Wenbo Pan, Zhichao Liu, Xianlong Wang, Haining Yu, and Xiaohua Jia. Towards long-horizon interpretability: Efficient and faithful multi-token attribution for reasoning LLMs. *arXiv preprint arXiv:2602.01914*, 2026. URL https://arxiv.org/abs/2602.01914.

Hugo Pitorro and Marcos Vinicius Treviso. LaTIM: Measuring latent token-to-token interactions in Mamba models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 24478–24493. Association for

Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.1194. URL https://aclanthology.org/2025.acl-long.1194/.

Farnoush Rezaei Jafari, Grégoire Montavon, Klaus-Robert Müller, and Oliver Eberle. MambaLRP: Explaining selective state space sequence models. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://papers.neurips.cc/paper_files/paper/2024/hash/d6d0e41e0b1ed38c76d13c9e417a8f1f-Abstract-Conference.html.

Avanti Shrikumar, Peyton Greenside, and Anshul Kundaje. Learning important features through propagating activation differences. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70, pp. 3145–3153. PMLR, 2017. URL https://proceedings.mlr.press/v70/shrikumar17a.html.

Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70, pp. 3319–3328. PMLR, 2017. URL https://proceedings.mlr.press/v70/sundararajan17a.html.

Richard S. Sutton, David A. McAllester, Satinder P. Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems*, volume 12, pp. 1057–1063, 1999. URL https://proceedings.neurips.cc/paper/1999/hash/464d828b85b0bed98e80ade0a5c43b0f-Abstract.html.

Songlin Yang and Yu Zhang. FLA: A triton-based library for hardware-efficient implementations of linear attention mechanism. Software repository, January 2024. URL https://github.com/fla-org/flash-linear-attention.

Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2412.06464.

A Formal Rules and Derivations

A.1 Finite propagation and contribution conservation

For an operation $v = f(u_1, \ldots, u_k)$, DeltaTrace constructs linear maps D_j from the two executions such that

$$\Delta v = \sum_{j=1}^{k} D_j \Delta u_j. \tag{7}$$

An upstream coefficient m_v propagates to input j as $D_j^\top m_v$. Coefficients from multiple consumers add at their shared input. Starting with coefficient 1 at the scalar target, the reverse traversal produces the source coefficients in Equation 2. For a scalar nonlinearity f, the local coefficient is the divided difference $(f(u_1) - f(u_0))/(u_1 - u_0)$, with its derivative as the coincident-input limit. Linear transformations retain their weights, and addition sends the coefficient to each summand.

Theorem 1 (Contribution conservation)**.** *For an acyclic computation over real-valued operations satisfying Equation 7, reverse propagation from a scalar target yields the conservation identity in Equation 3.*

Proof. An acyclic computation can be expanded into elementary operations with explicit branches. At an operation $v = f(u_1, \ldots, u_k)$, Equation 7 gives

$$\langle m_v, \Delta v \rangle = \sum_j \left\langle D_j^\top m_v, \Delta u_j \right\rangle. \tag{8}$$

Replacing a node's contribution by the contributions of its parents therefore preserves the total on a cut through the graph. At shared parents, coefficients from different consumers sum. Repeated substitution reaches the input cut and yields Equation 3, since the coefficient of the scalar target is 1. Constant inputs have zero difference and contribute zero to this sum.

□

A.2 Finite attention normalization

For an attention row $p_b = \mathrm{softmax}(z_b)$, $b \in \{0, 1\}$, the logarithmic mean and its sum are

$$\ell_i = \frac{p_{1,i} - p_{0,i}}{\log p_{1,i} - \log p_{0,i}}, \qquad \tau = \sum_i \ell_i, \tag{9}$$

with $\ell_i = p_{0,i}$ at coincident probabilities. All quantities are evaluated on the common unmasked support.

Proposition 1 (Finite attention normalization)**.** *For finite logits on a common support,*

$$\Delta p = \left(\mathrm{diag}(\ell) - \frac{\ell \ell^\top}{\tau} \right) \Delta z. \tag{10}$$

Proof. The common support contains positive probabilities. For endpoint b, $\log p_b = z_b - a_b \mathbf{1}$, where $a_b = \log \sum_i \exp(z_{b,i})$. The definition of the logarithmic mean gives

$$\Delta p = \ell \odot (\Delta z - \Delta a \, \mathbf{1}). \tag{11}$$

Probability normalization implies $\mathbf{1}^\top \Delta p = 0$, so

$$\Delta a = \frac{\ell^\top \Delta z}{\tau}. \tag{12}$$

Substitution proves Equation 10. For a target coordinate y,

$$\Delta \log p_y = \left(e_y - \frac{\ell}{\tau} \right)^\top \Delta z. \tag{13}$$

The coincident-probability definition follows from the continuous limit of the logarithmic mean. The score operator is symmetric positive semidefinite: for $w_i = \ell_i/\tau$,

$$u^\top \left(\mathrm{diag}(\ell) - \frac{\ell\ell^\top}{\tau} \right) u = \tau \left(\sum_i w_i u_i^2 - \left(\sum_i w_i u_i \right)^2 \right) \geq 0. \tag{14}$$

This is the weighted variance of u multiplied by τ.

□

The operator is symmetric. Its transpose action gives Equation 5, and the response coefficient for a log-softmax target is $m_z = e_y - \ell/\tau$.

A.3 Shared operator rules

For attention output $Y = PV$, Equation 4 gives the coefficients $m_V = P_1^\top U$ and $m_P = UV_0^\top$ for an upstream coefficient U.

Query–key scores use the symmetric product identity,

$$\Delta(QK^\top) = \Delta Q\, \bar{K}^\top + \bar{Q}\, \Delta K^\top, \qquad \bar{Q} = (Q_0 + Q_1)/2, \tag{15}$$

with $\bar{K}$ defined in the same way. The common operator rules appear in Table 4. They depend on the operator and its role, and are shared by the two model families. Normalization, rotary position transformations, and grouped heads follow their respective model definitions.

Table 4: Fixed propagation rules shared across architectures. Bars denote endpoint averages. Gated-memory operations extend the same content-oriented rule to recurrent states.

Operation	Change decomposition
Linear and residual	$\Delta(Wx + b) = W\Delta x$; $\Delta(a + b) = \Delta a + \Delta b$
Attention values	$\Delta(PV) = P_1\Delta V + \Delta P V_0$
Attention scores	$\Delta(QK^\top) = \Delta Q\bar{K}^\top + \bar{Q}\Delta K^\top$
SwiGLU	$\Delta(us) = \bar{s} \odot \Delta u + \bar{u} \odot \Delta s$, $s = \mathrm{SiLU}(g)$
Output gating	$\Delta(ns) = s_1 \odot \Delta n + n_0 \odot \Delta s$
RMS normalization	Analytic finite rule for $wx/\sqrt{\mathrm{mean}(x^2) + \epsilon}$

A.4 Gated-memory recurrence

With column vectors and a key-by-value state $S_t \in \mathbb{R}^{d_k \times d_v}$, its per-token computation is

$$\begin{aligned} T_t &= \alpha_t S_{t-1}, & r_t &= T_t^\top k_t, \\ c_t &= v_t - r_t, & u_t &= \beta_t c_t, \\ S_t &= T_t + k_t u_t^\top, & o_t &= S_t^\top q_t. \end{aligned} \tag{16}$$

The query includes the model's fixed attention scale. The state T_t contains retained evidence, r_t reads existing content, and u_t writes the update. DeltaTrace follows the change of this computation using fixed content-oriented products:

$$\begin{aligned} \Delta T_t &= \alpha_{t,1}\Delta S_{t-1} + S_{t-1,0}\Delta\alpha_t, \\ \Delta r_t &= (\Delta T_t)^\top k_{t,1} + T_{t,0}^\top \Delta k_t, \\ \Delta u_t &= \beta_{t,1}(\Delta v_t - \Delta r_t) + c_{t,0}\Delta\beta_t, \\ \Delta S_t &= \Delta T_t + k_{t,1}\Delta u_t^\top + \Delta k_t\, u_{t,0}^\top, \\ \Delta o_t &= (\Delta S_t)^\top q_{t,1} + S_{t,0}^\top \Delta q_t. \end{aligned} \tag{17}$$

This recurrence assigns contributions to new values, retrieval keys, queries, and both gates. Reversing it carries the response coefficient through stored evidence to earlier sources. Scalar gate nonlinearities and query/key normalization use the same finite propagation construction as the surrounding network.

A.5 Product and normalization identities

For a bilinear map B, both

$$\Delta B(a,b) = B(\Delta a, \bar{b}) + B(\bar{a}, \Delta b), \tag{18}$$

$$\Delta B(a,b) = B(\Delta a, b_1) + B(a_0, \Delta b) \tag{19}$$

follow by expansion. Equation 4, the shared SwiGLU rule, and every product in Equation 17 are direct instances with the stated argument ordering. Subtraction and state addition are linear, completing the finite memory recurrence.

For RMS normalization $f(x) = w \odot x/r(x)$, $r(x) = \sqrt{\|x\|^2/d + \epsilon}$, the symmetric product rule gives an analytic coefficient. With $\bar{x} = (x_0 + x_1)/2$, $r_b = r(x_b)$, and $v = w \odot m$, its transpose action is

$$D_f^\top m = \frac{r_0^{-1} + r_1^{-1}}{2}\, v - \frac{2\bar{x}\ \langle \bar{x}, v\rangle}{d\, r_0 r_1 (r_0 + r_1)}. \tag{20}$$

The identity follows from $\Delta r = (2\langle \bar{x}, \Delta x\rangle/d)/(r_0 + r_1)$ and $\Delta(r^{-1}) = -\Delta r/(r_0 r_1)$. The same construction covers fixed learned scale parameters and the query/key normalization used in the memory path.

B Implementation and Storage Accounting

Attention. Native endpoint queries, keys, values, and log-normalizers determine the probabilities within each FlashAttention tile. The first sweep accumulates the logarithmic-mean normalizers and weighted centers in Equation 5. Subsequent sweeps accumulate query, key, and value coefficients. Endpoint averages reside in linear-size buffers or are computed inside the tiles.

Memory. The implementation processes 64-token chunks using FLA's native chunk-state adjoints and batched contractions with saved endpoint states. An associative affine scan computes the finite decay contribution. Convolution and surrounding projections use the installed model primitives and their linear transpose actions.

Storage. For sequence length T, head dimension d, and tile or chunk width C, attention vectors and row statistics occupy $O(Td)$ storage per head; pairwise products reside in bounded tiles. Dense attention arithmetic is $O(T^2 d)$. Chunked memory occupies $O(Td + TC + (T/C)d^2)$ auxiliary storage per head for token coefficients, within-chunk products, and boundary states. Both storage expressions are linear in T at fixed d and C. Layer replay supplies intermediate activations, with checkpoint storage accounted for separately.

Execution. Qwen3 uses FP16; Qwen3.5 uses BF16 attention and FLA, with FP32 accumulation where supplied by the underlying operations. Complete attribution cost includes the two endpoint executions, layer replay, contribution propagation, and transfers.

C Evaluation Specification

The FlashTrace `table1-data-v1` release fixes prompt transformation, eligible source positions, complete response targets, gold mapping, and scoring. Qwen3-8B uses the frozen `clean-v1-20260909` method and the `clean-v1-eval3-native-batch-20260909` evaluation entry point. All 13 tasks are complete. Attribution uses native FlashAttention with one real example and its two endpoints per call; response scoring uses the original eager implementation. The runtime record identifies a MetaX C550 64GB device, PyTorch 2.8.0, and Transformers 4.57.3.

Metric views. For a signed source vector A, RISE uses descending signed scores A_i, while MAS and released NIAH recovery use $\max(A_i, 0)$. The original evaluator produces 21 response values across 20 deletions. Saved signed deletion curves supply RISE; when the positive and signed views have an identical ranked prefix until the monotone normalized response reaches zero, the recorded prefix proof permits reuse of that RISE value. MAS always comes from the positive view. The

illustrative curves in Figures 4 and 5 retain positive-ranking DT deletion and add the matching one-hop FT curve for each model. The score sums token log-probabilities over the entire fixed response plus EOS. Within each model and example, DT and FT share identical full-input and fully-deleted scores. The released evaluator normalizes by these endpoints, clips to $[0, 1]$, and takes the cumulative minimum.

Published reference results. For RISE, MAS, and NIAH recovery, the published baselines reproduce the authors' released task CSVs and original Tables 1 and 2. Each value is matched back to its CSV and aggregation row at full precision: these use the row-attribution mean, with the published IFR MQ-Q8 faithfulness pair using the recursive-attribution mean. The original fast perturbation variants are retained outside MQ-Q2 and MoreHopQA. All 95 additional source CSVs are verified against the released archive, and all 160 added metric values agree with the printed tables to their reporting precision. Baseline target aggregation follows the publication; deletion faithfulness evaluates the complete stored response. Their accompanying per-example `n1` trace records reproduce the CSV means; input text, target, eligible positions, and gold mappings were checked against each paired DELTATRACE example. The separately executed development comparison uses one recursive hop for faithfulness and three for recovery, as stated in Table 5. Qwen3.5 uses the same fixed inputs and responses with both methods evaluated on the same model.

VT and HotpotQA recovery. We re-evaluate five baselines on the same 448 inputs and reuse verified DELTATRACE/FT K3 vectors. VT reconstructs the cached answer without its reasoning prefix and ranks positive eligible body-token scores. HotpotQA retains the full response, sums signed scores over native body sentences, and retrieves the longest ranking prefix within the all-body-token budget, including punctuation and whitespace. Scopes and task budgets were selected retrospectively; the full budget sweep is retained in the experimental records. Perturbation, REAGENT, and CLP use 20 source segments. AttnLRP† includes a zero-ratio numeric repair and lossless activation offload.

Table 5: Paired development results: the first eight MQ-Q2 and eight MoreHopQA examples per model. These 32 model–example pairs form a separate development set. FT uses one recursive hop for RISE/MAS and three for recovery. RISE is signed; MAS and recovery use positive scores.

Model	Task	Method	RISE ↓	MAS ↓	Recovery@10% ↑
Qwen3-8B	MQ-Q2	FlashTrace	0.0685	0.1547	56.45
		DELTATRACE	0.0574	0.0806	77.52
	MoreHopQA	FlashTrace	0.1111	0.1969	—
		DELTATRACE	0.0585	0.1173	—
Qwen3.5-9B	MQ-Q2	FlashTrace	0.0942	0.2485	63.92
		DELTATRACE	0.1115	0.1518	73.35
	MoreHopQA	FlashTrace	0.2356	0.2797	—
		DELTATRACE	0.1688	0.2426	—

D IMPLEMENTATION COST MEASUREMENTS

The cost records retain each benchmark's implementation snapshot and source hashes. The Qwen3 pairwise and tiling benchmarks use the native finite-P1 implementation recorded on 7 September, preceding the frozen full-task evaluation. The Qwen3.5 batching and replay-retention benchmarks use their recorded 9 September snapshots. Each experiment compares its alternatives on the same inputs within one benchmark run. Input and response tokens are both included in sequence length. Complete attribution includes input preparation, capture, replay, propagation, projection, diagnostics, and returning source scores; model loading, warmup, and deletion scoring have separate records.

Paired Qwen3 cost. Every method has one warm call followed by three measured calls in rotated order on each of 16 development examples. Per-example medians are aggregated by task in Figure 2. FlashTrace uses the authors' eager implementation, while DELTATRACE uses native FlashAttention

and finite propagation. The maximum allocated memory includes model weights. Three-hop FlashTrace takes 18.27 seconds for the same 16 examples; this is a separate recursion setting from the one-hop faithfulness comparison.

Tiling on longer released examples. Figure 3 compares dense and tiled finite propagation on the 1,241-token retrieval control, the longest released HotpotQA context (3,470 total tokens), and the longest MATH rollout (3,762 total tokens). These are actual released examples selected by input or rollout length. At 3,470 tokens, tiling reduces complete attribution time from 5.32 to 3.02 seconds and peak allocation from 32.66 to 22.54 GB. At 3,762 tokens, the corresponding values are 6.12 to 3.40 seconds and 37.18 to 30.06 GB. Full source-vector relative L_2 differences between the dense and tiled implementations range from 4.46×10^{-4} to 2.31×10^{-3} on these examples.

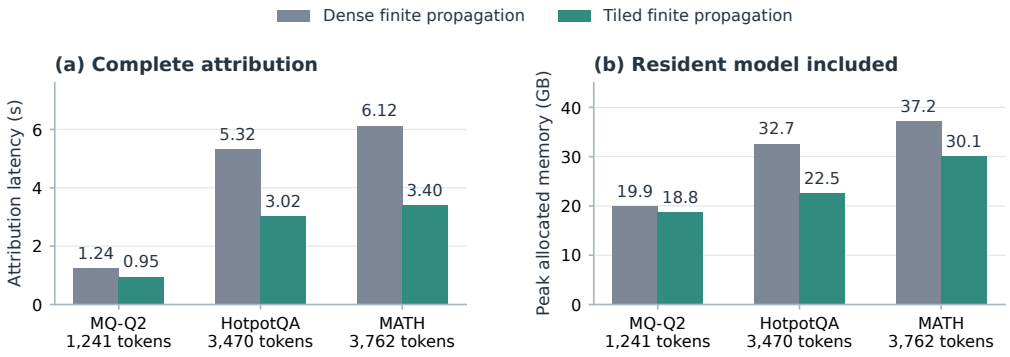

Figure 3: **Dense and tiled finite propagation on three released Qwen3-8B examples.** Complete warm attribution latency is the median of repeated calls. Memory is peak allocated device memory, including resident weights. Each input keeps its original prompt and full stored response. The three groups are distinct tasks and response lengths; bars show the measured examples.

Real batching and replay retention. For Qwen3.5, 16 examples are sorted by total length and paired before attribution, then evaluated in two interleaved warm rounds. Serial B1 and actual B2 use the same complete examples; internal endpoint batches are two and four, respectively. Shape warmup takes 69.74 and 46.53 seconds in the recorded process, separately from the 16.68 and 12.33 second warm passes. Figure 2 uses this exact pairing. GPU checkpoints and batching change floating-point execution; the recorded B1/B2 source-vector relative L_2 differences are approximately 0.013–0.026. Quality remains attached to the corresponding execution record. Subsequent replay-retention measurements fix the batching plan, preserve every paired complete vector, and reduce warm time by 5.80% on Qwen3 and 5.15% on Qwen3.5 (Table 6). These comparisons measure each change against its own recorded baseline.

Table 6: Additional paired replay-retention measurements on 16 development examples. Each row compares the existing implementation with retained replay within one benchmark. Warm time is the mean complete pass over 16 examples across two interleaved rounds. Peak memory gives before/after allocations, including resident weights. All paired complete source vectors were identical. The baselines already include prior scheduling improvements.

Model	Sample batch	Before (s)	Retained (s)	Reduction	Peak GB (before/after)
Qwen3-8B	1	10.360	9.759	5.80%	18.76 / 18.67
Qwen3.5-9B	2	11.480	10.889	5.15%	23.63 / 23.63

E Detailed Signed Evidence Views

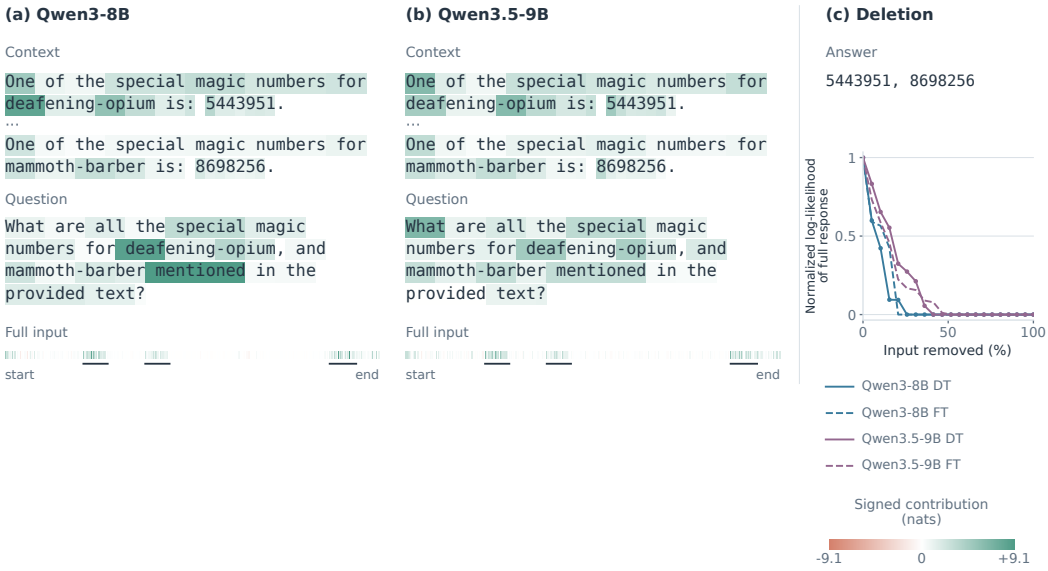

Figure 4: **Signed evidence for retrieving two numbers.** (a,b) The same `niah_mq_q2` input on Qwen3-8B and Qwen3.5-9B; teal and coral show positive and negative DT contributions in nats. Marks locate excerpts. (c) Each model's DT (solid) and one-hop FT (dashed) curves show the normalized log-likelihood of the entire fixed response plus EOS under 20-step deletion. Color identifies the model. The released normalization uses that model's full-input and fully-deleted scores, clipping and a cumulative minimum. DT deletion uses positive ranking. Answer summarizes the full stored response.

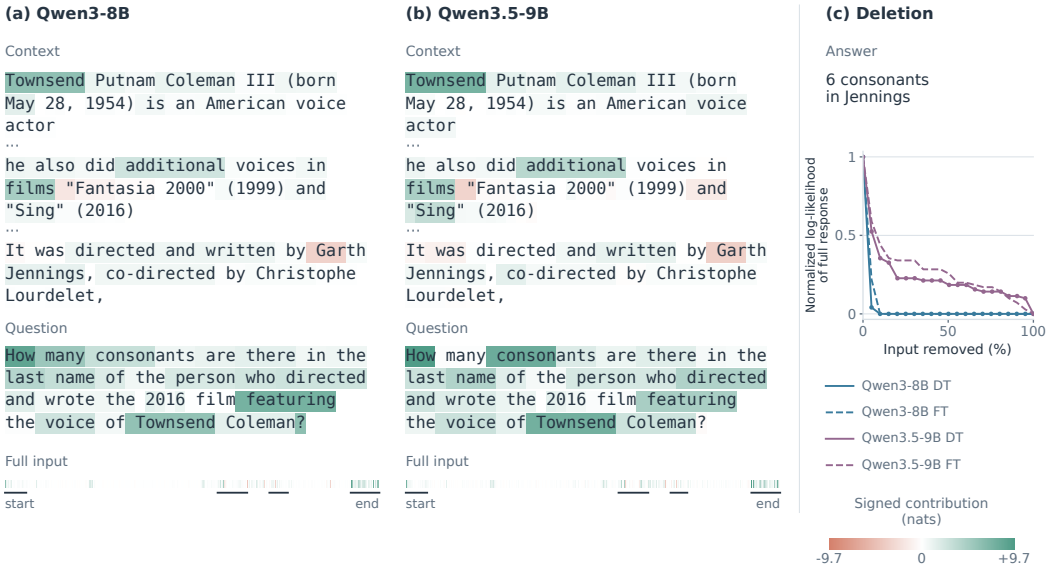

Figure 5: **Signed evidence across a multi-hop question.** (a,b) The MoreHopQA input links Townsend Coleman to *Sing*, Garth Jennings, and his surname's consonants. (c) Normalized log-likelihood of the entire fixed response plus EOS, comparing each model's DT with its own one-hop FT baseline. Model colors, method line styles, and per-model normalization follow Figure 4. Both heatmaps share the signed scale within this example.

F Counterfactual Credit and Policy Learning

The response attribution studied here suggests a broader research direction: propagating finite reward differences to actions that contribute to them. The connection to reinforcement learning is particularly direct when counterfactual returns are averaged exactly. The following identity expresses the familiar action-independent-baseline principle (Sutton et al., 1999) in counterfactual form. COMA uses the related construction to assign credit by averaging one agent's action while fixing the others (Foerster et al., 2018).

Let h be a decision history and π a fixed continuation policy. Write $G^\pi(h, a; \xi)$ for the integrable return obtained by setting the current action to a and subsequently following π, with ξ supplying environment and policy randomness. Define

$$Q^\pi(h, a) = \mathbb{E}_\xi[G^\pi(h, a; \xi)], \qquad V^\pi(h) = \mathbb{E}_{a' \sim \pi(\cdot|h)}[Q^\pi(h, a')]. \tag{21}$$

The reference action a' is sampled independently of the selected action a, conditional on h. The two rollouts may share randomness, with each retaining its correct marginal distribution under the corresponding action intervention.

Proposition 2 (Exactly averaged counterfactual credit)**.** *Under these definitions, the policy-averaged counterfactual credit satisfies*

$$C^\pi(h, a) = \mathbb{E}_{a', \xi}[G^\pi(h, a; \xi) - G^\pi(h, a'; \xi)] = Q^\pi(h, a) - V^\pi(h). \tag{22}$$

For a differentiable policy π_θ with positive probabilities over a fixed finite action set, the score-function learning signals obey

$$\mathbb{E}_{a \sim \pi_\theta}[\nabla_\theta \log \pi_\theta(a \mid h)\, C^{\pi_\theta}(h, a)] = \mathbb{E}_{a \sim \pi_\theta}[\nabla_\theta \log \pi_\theta(a \mid h)\, Q^{\pi_\theta}(h, a)]\,. \tag{23}$$

Proof. Linearity of expectation gives Equation 22. The baseline term in Equation 23 is

$$V^{\pi_\theta}(h) \sum_a \pi_\theta(a \mid h) \nabla_\theta \log \pi_\theta(a \mid h) = V^{\pi_\theta}(h) \nabla_\theta \sum_a \pi_\theta(a \mid h) = 0. \tag{24}$$

Subtracting this term proves the result. Credit and action value serve as weights in the score-function update. □

This equality provides a concrete target for extending finite attribution to learning: a reward-level construction whose averaged action credit recovers the advantage. The resulting research questions concern how computational or environmental structure can make this credit efficient to estimate, how reference choices and coupled rollouts affect its variance, and how the estimate behaves when a trajectory is extended by steps irrelevant to the reward. These questions connect signed attribution to the quality and temporal stability of a learning signal.